

EN3160: Image Processing and Computer Vision

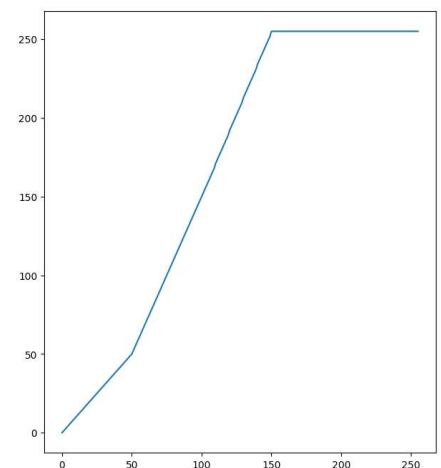
Assignment 01: Intensity Transformations and Neighborhood Filtering

- **Name:** Ayanaja N.B.G.M
- **Index Number:** 230065H
- **GitHub Repository:** https://github.com/ManiiAya/EN3160_Assignment_01

Question 1: Piecewise Linear Intensity Transformations

```
In [237]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
```

```
In [238]: x = [0,50,100,150,255]
y = [0,50,150,255,255]
f = cv.imread("images/emma.jpg", cv.IMREAD_GRAYSCALE)
X = np.arange(256, dtype='uint8')
F = np.interp(X,x,y).astype(np.uint8)
fig, axes = plt.subplots(1,3, figsize=(24,8))
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[1].imshow(F[f], cmap='gray', vmin=0, vmax=255)
axes[2].plot(X,F)
plt.show()
```



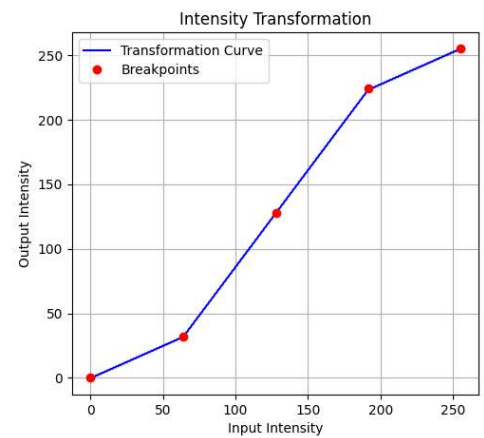
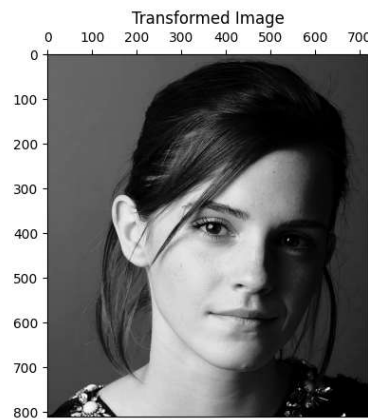
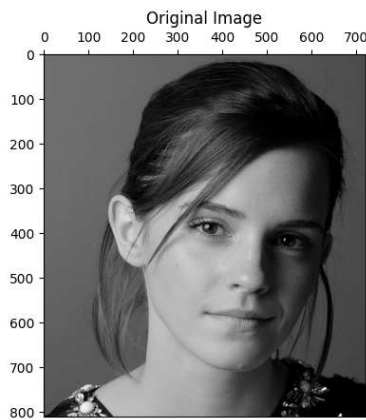
```
In [239]: def intensity_transform(im, breakpoints):
x, y = zip(*breakpoints)
X = np.arange(256, dtype='uint8')
F = np.interp(X, x, y).astype(np.uint8)
return F[im], X, F
```

```
In [240]: f = cv.imread("images/emma.jpg", cv.IMREAD_GRAYSCALE)
breakpoints = [(0, 0), (64, 32), (128, 128), (192, 224), (255, 255)]
transformed_img, X, F = intensity_transform(f, breakpoints)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image");axes[0].xaxis.set_ticks_position('top')
axes[1].imshow(transformed_img, cmap='gray', vmin=0, vmax=255)
axes[1].set_title("Transformed Image");axes[1].xaxis.set_ticks_position('top') # Hides the axis
axes[2].plot(X, F, color='blue', label='Transformation Curve')
x_coords, y_coords = zip(*breakpoints)
```

```

axes[2].plot(x_coords, y_coords, 'ro', label='Breakpoints')
axes[2].set_title("Intensity Transformation")
axes[2].set_xlabel("Input Intensity")
axes[2].set_ylabel("Output Intensity")
axes[2].grid(True)
axes[2].legend()
plt.tight_layout()
plt.show()

```

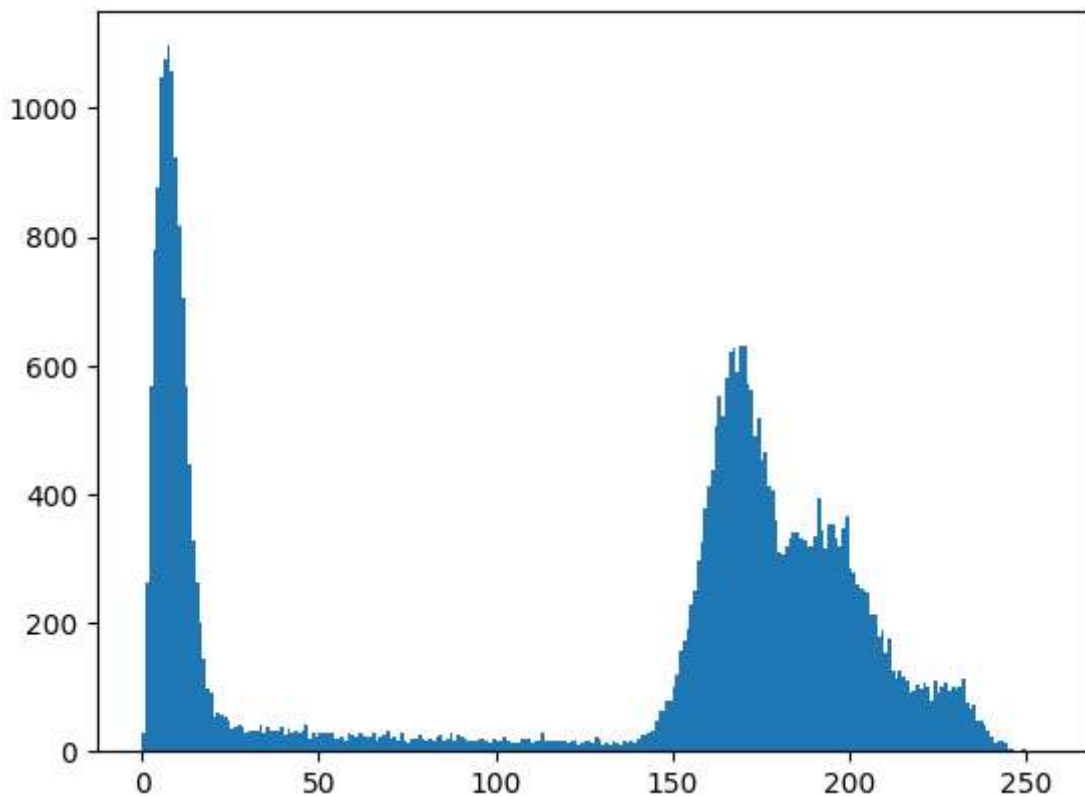


Question 2: Feature Accentuation in Medical Imagery

```

In [241]: brain_img = cv.imread("images/brain.png", cv.IMREAD_GRAYSCALE)
hist_values, bin_edges, _ = plt.hist(brain_img.ravel(), bins=256, range=[0,256])
plt.show()
#printed the histogram values for better understanding of the distribution of pixel intensities
# for intensity in range(140,256):
#     print(f"Intensity {intensity}: {100*hist_values[intensity]/np.max(hist_values)}")

```



```

In [242]: f = cv.imread("images/brain.png", cv.IMREAD_GRAYSCALE)
breakpoints = [(0, 0), (153, 0), (154, 100), (168, 255), (184, 100), (185, 0), (221, 0)]

```

```

transformed_img, X, F = intensity_transform(f, breakpoints)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

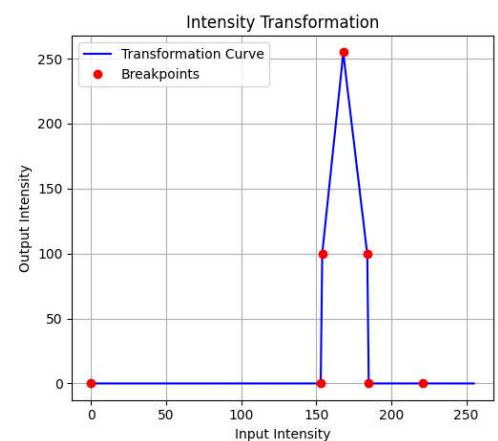
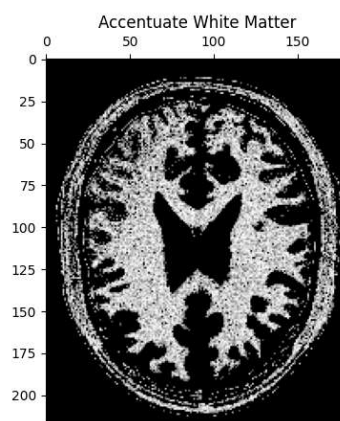
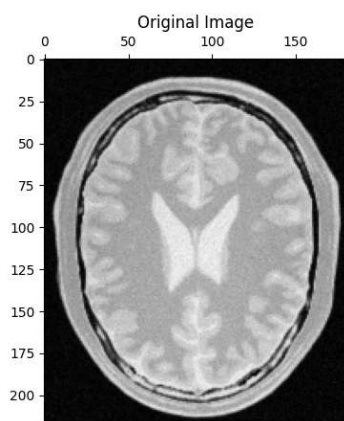
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image")
axes[0].xaxis.set_ticks_position('top')

axes[1].imshow(transformed_img, cmap='gray', vmin=0, vmax=255)
axes[1].set_title("Accentuate White Matter")
axes[1].xaxis.set_ticks_position('top')

axes[2].plot(X, F, color='blue', label='Transformation Curve')
x_coors, y_coors = zip(*breakpoints)
axes[2].plot(x_coors, y_coors, 'ro', label='Breakpoints')
axes[2].set_title("Intensity Transformation")
axes[2].set_xlabel("Input Intensity")
axes[2].set_ylabel("Output Intensity")
axes[2].grid(True)
axes[2].legend()

plt.tight_layout()
plt.show()

```



```

In [243]: f = cv.imread("images/brain.png", cv.IMREAD_GRAYSCALE)
breakpoints = [(0, 0), (185, 0), (186, 100), (194, 255), (221, 100), (222, 0), (255, 0)]

transformed_img, X, F = intensity_transform(f, breakpoints)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

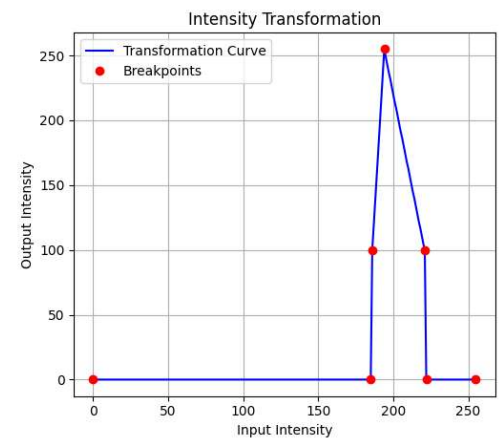
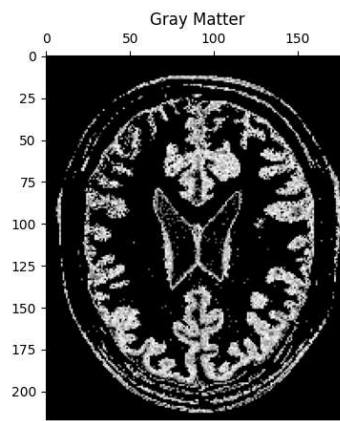
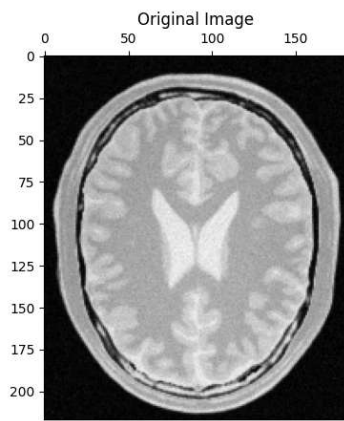
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image")
axes[0].xaxis.set_ticks_position('top')

axes[1].imshow(transformed_img, cmap='gray', vmin=0, vmax=255)
axes[1].set_title("Gray Matter")
axes[1].xaxis.set_ticks_position('top')

axes[2].plot(X, F, color='blue', label='Transformation Curve')
x_coors, y_coors = zip(*breakpoints)
axes[2].plot(x_coors, y_coors, 'ro', label='Breakpoints')
axes[2].set_title("Intensity Transformation")
axes[2].set_xlabel("Input Intensity")
axes[2].set_ylabel("Output Intensity")
axes[2].grid(True)
axes[2].legend()

plt.tight_layout()
plt.show()

```



Interpretation of Question 2: Tissue Accentuation

To analyze specific tissues in the brain proton density image, I first plotted the histogram to identify the distinct intensity clusters.

- **White Matter:** On the histogram, I identified this as the first major peak. I designed a triangular breakpoint filter ranging from 153 to 185, peaking at an intensity of **168**.
- **Gray Matter:** identified this as the second peak on the histogram and designed a triangular breakpoint filter ranging from 185 to 222, peaking at an intensity of **194**.

So by applying these custom breakpoint coordinates, the target matter is driven to 255 (white) while all other background and tissue values are suppressed to 0 (black).

Question 3: Gamma Correction in Lab Color Space

```
In [244]: img_original = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\white.jpg", cv.IMREAD_COLOR)
f = cv.cvtColor(img_original, cv.COLOR_BGR2LAB)
L,a,b = cv.split(f)
gamma = 0.5
Lc = (((L/255.0)**(gamma))*255).astype(np.uint8)
img_lab = cv.merge([Lc,a,b])
img = cv.cvtColor(img_lab, cv.COLOR_LAB2RGB)
fig, axes = plt.subplots(2, 2, figsize=(12, 9))
axes[0, 0].imshow(cv.cvtColor(img_original, cv.COLOR_BGR2RGB))
axes[0, 0].set_title("Original Image")
axes[0, 0].axis('off')
axes[0, 1].imshow(img)
axes[0, 1].set_title(f"Gamma Corrected Image (Gamma = {gamma})")
axes[0, 1].axis('off')
axes[1, 0].hist(L.ravel(), bins=256, range=[0, 256], color='gray')
axes[1, 0].set_title("Histogram: Original L Plane")
axes[1, 0].set_xlabel("Pixel Intensity")
axes[1, 0].set_ylabel("Frequency")
axes[1, 1].hist(Lc.ravel(), bins=256, range=[0, 256], color='gray')
axes[1, 1].set_title(f"Histogram: Corrected L Plane (Gamma = {gamma})")
axes[1, 1].set_xlabel("Pixel Intensity")
axes[1, 1].set_ylabel("Frequency")

plt.tight_layout()
plt.show()
```

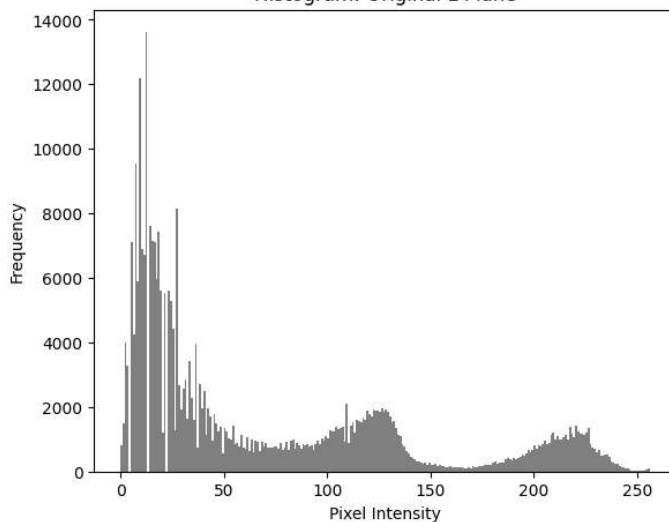
Original Image



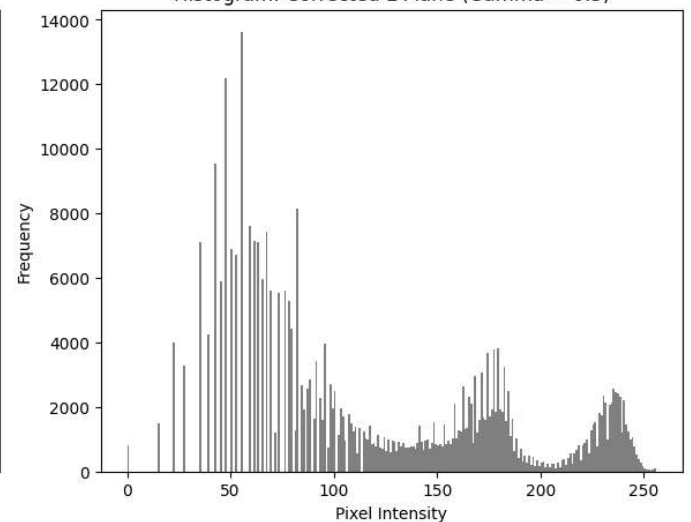
Gamma Corrected Image (Gamma = 0.5)



Histogram: Original L Plane



Histogram: Corrected L Plane (Gamma = 0.5)



Question 4: Vibrance Enhancement via Saturation Transformation

```
In [245]: f = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\spider.png", cv.IMREAD_COLOR)
img = cv.cvtColor(f, cv.COLOR_BGR2HSV)
h,s, v = cv.split(img)
sigma = 70
a = 0.45
i = np.arange(256)
F = i + a * (128 * np.exp(-((i-128)**2) / (2*sigma**2)))
F = np.clip(F,0,255).astype(np.uint8)
s_new = F[s]
img_new = cv.merge([h,s_new,v])
img_rgb = cv.cvtColor(img_new, cv.COLOR_HSV2RGB)
fig, axes = plt.subplots(1,3, figsize=(19,4))
axes[0].plot(i, F);axes[0].set_title("Vibrance Transformation Curve")
axes[0].set_xlabel("Input Saturation");axes[0].set_ylabel("Output Saturation")
axes[0].grid(True) # Adds a grid for better readability
axes[1].imshow(cv.cvtColor(f, cv.COLOR_BGR2RGB)); axes[1].xaxis.set_ticks_position('top') ;
axes[1].set_title("Original Image")
axes[2].imshow(img_rgb); axes[2].xaxis.set_ticks_position('top')
axes[2].set_title(f"Transformed Image - a: {a}")
plt.tight_layout()
plt.show()
```



To achieve a visually pleasing output that enhances the mid-tone colors of the image without oversaturating the bright reds, I adjusted the parameter to $a = 0.45$

Question 5: Global Histogram Equalization

```
In [246]: def equalize(f):
            hist, _ = np.histogram(f.flatten(),256,[0,256],True)
            cum_sum = np.cumsum(hist)*(255)
            F = cum_sum.astype(np.uint8)
            img = F[f]
            return img

f =cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\shells.tif", cv.IMREAD_GRAYSCALE)
img_eq = equalize(f)
fig, axes = plt.subplots(1, 4, figsize=(20, 5))

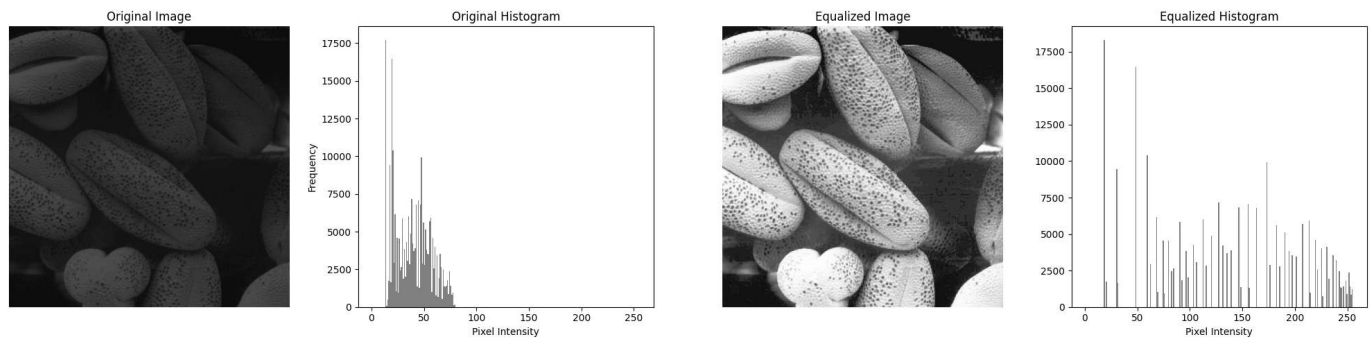
# 1. Original Image
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image")
axes[0].axis("off")

# 2. Original Histogram
axes[1].hist(f.flatten(), bins=256, range=[0, 256], color='gray')
axes[1].set_title("Original Histogram")
axes[1].set_xlabel("Pixel Intensity")
axes[1].set_ylabel("Frequency")

# 3. Equalized Image
axes[2].imshow(img_eq, cmap='gray', vmin=0, vmax=255)
axes[2].set_title("Equalized Image")
axes[2].axis("off")

# 4. Equalized Histogram
axes[3].hist(img_eq.flatten(), bins=256, range=[0, 256], color='gray')
axes[3].set_title("Equalized Histogram")
axes[3].set_xlabel("Pixel Intensity")

plt.tight_layout()
plt.show()
```



Question 6: Foreground-Aware Histogram Equalization

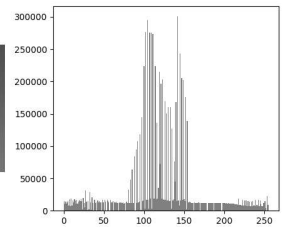
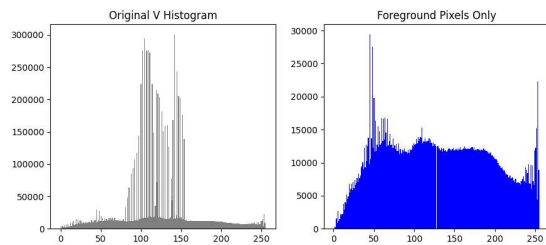
```
In [247]: f = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\jeniffer.jpg", cv.IMREAD_COLOR)
h, s, v = cv.split(cv.cvtColor(f, cv.COLOR_BGR2HSV))
fig, axes = plt.subplots(1,3, figsize=(15,6))
axes[0].imshow(h, cmap='gray', vmin=0, vmax=255); axes[0].xaxis.set_ticks_position('top'); axes
axes[1].imshow(s, cmap='gray', vmin=0, vmax=255); axes[1].axis(False); axes[1].set_title("Satura
axes[2].imshow(v, cmap='gray', vmin=0, vmax=255); axes[2].axis(False); axes[2].set_title("Value"
plt.tight_layout()
plt.show()
```



By observing above three graphs, it can be concluded that the Saturation plane is optimal to mask the foreground.

```
In [248]: _, mask = cv.threshold(s, 11, 255, cv.THRESH_BINARY)
foreground = cv.bitwise_and(v,v, mask=mask)
hist, _ = np.histogram(foreground[mask==255],256,[0,256],density=True)
cum_sum = np.cumsum(hist)*(255)
F = cum_sum.astype(np.uint8)
v_eq = v.copy()
v_eq[mask==255]= F[v[mask==255]]
img_rgb_eq = cv.cvtColor(cv.merge([h,s,v_eq]), cv.COLOR_HSV2RGB)
print(F.shape)
fig, axes = plt.subplots(1, 5, figsize=(22, 4))
axes[0].hist(v.flatten(), bins=256, color='gray')
axes[0].set_title("Original V Histogram")
axes[1].hist(v[mask==255].ravel(), bins=256, color='blue')
axes[1].set_title("Foreground Pixels Only")
axes[2].imshow(cv.cvtColor(f, cv.COLOR_BGR2RGB))
axes[2].set_title("Original Image")
axes[2].axis("off")
axes[3].imshow(img_rgb_eq)
axes[3].set_title("Foreground Equalized Image")
axes[3].axis("off")
axes[4].hist(v_eq.flatten(), bins=256, color='gray')
plt.tight_layout()
plt.show()
```

(256,)



```
In [249]: plt.imshow(mask, cmap='gray')
plt.title("Foreground Mask")
plt.axis("off")
```

```
Out[249]: (np.float64(-0.5), np.float64(3839.5), np.float64(2159.5), np.float64(-0.5))
```

Foreground Mask



Question 7: Edge Detection via Sobel Operator and Kernel Separability

```
In [250]: f = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\ein.png", cv.IMREAD_REDUCED_GRAYSC
print(f.shape)
sobel_kernel = np.array([[ 1, 0, -1],
                          [ 2, 0, -2],
                          [ 1, 0, -1]])

sobel_horizontal_kernel = np.array([[ 1, 2, 1],
                                     [ 0, 0, 0],
                                     [-1, -2, -1]])

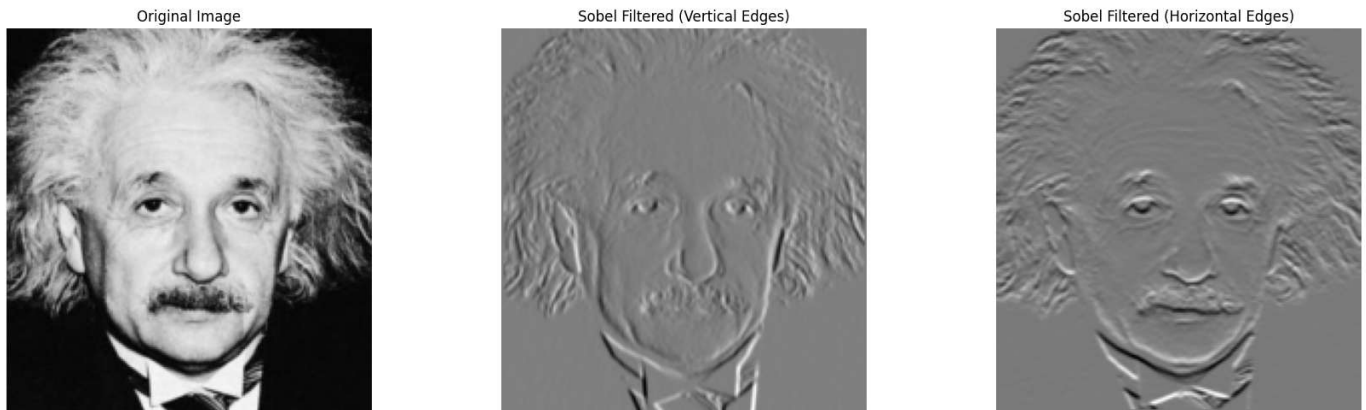
img_filtered_horizontal = cv.filter2D(f, cv.CV_64F, sobel_horizontal_kernel)
img_filtered = cv.filter2D(f, cv.CV_64F, sobel_kernel)
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image")
axes[0].axis("off")

axes[1].imshow(img_filtered, cmap='gray')
axes[1].set_title("Sobel Filtered (Vertical Edges)")
axes[1].axis("off")

axes[2].imshow(img_filtered_horizontal, cmap='gray')
axes[2].set_title("Sobel Filtered (Horizontal Edges)")
axes[2].axis("off")
```

```
plt.tight_layout()
plt.show()
```

(190, 182)



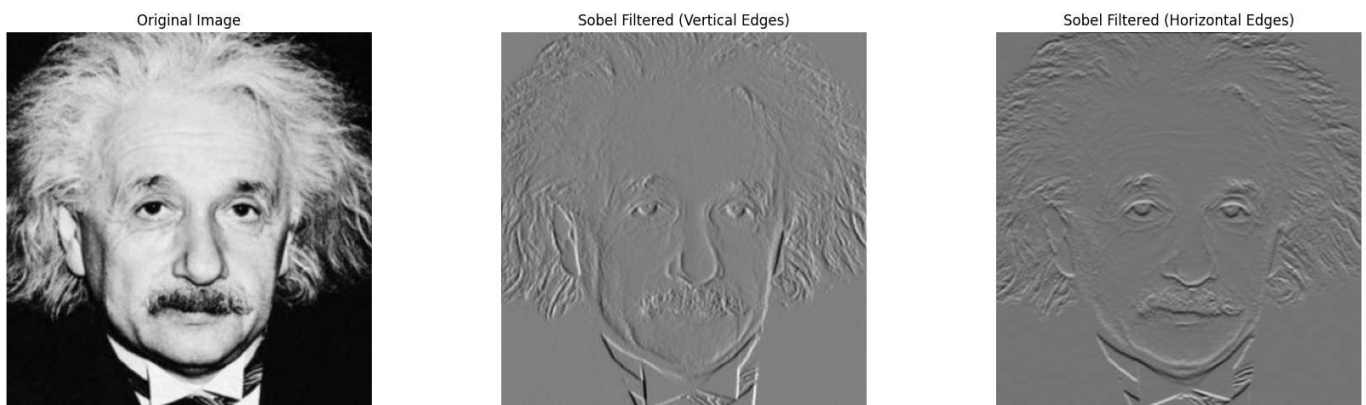
```
In [251]: def sobel(img, kernel):
            rows, cols = img.shape
            output = np.zeros((rows, cols), dtype=np.float64)
            pad_img = np.pad(img, pad_width=1, mode='constant', constant_values=0)
            for i in range(rows):
                for j in range(cols):
                    region = pad_img[i:i+3, j:j+3]
                    output[i, j] = np.sum(region * kernel)
            return output

f = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\ein.png", cv.IMREAD_GRAYSCALE)
sobel_result = sobel(f, sobel_kernel)
sobel_horizontal_result = sobel(f, sobel_horizontal_kernel)
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image")
axes[0].axis("off")

axes[1].imshow(sobel_result, cmap='gray')
axes[1].set_title("Sobel Filtered (Vertical Edges)")
axes[1].axis("off")

axes[2].imshow(sobel_horizontal_result, cmap='gray')
axes[2].set_title("Sobel Filtered (Horizontal Edges)")
axes[2].axis("off")

plt.tight_layout()
plt.show()
```



```
In [252]: v_kernel = np.array([[1],
                                [2],
                                [1]])
```

```

h_kernel = np.array([[1, 0, -1]])

h_kernel2 = np.array([[1], [0], [-1]]) # vertical difference
v_kernel2 = np.array([[1, 2, 1]])
conv2 = cv.filter2D(f, cv.CV_64F, v_kernel2)
img_filtered_horizontal = cv.filter2D(conv2, cv.CV_64F, h_kernel2)

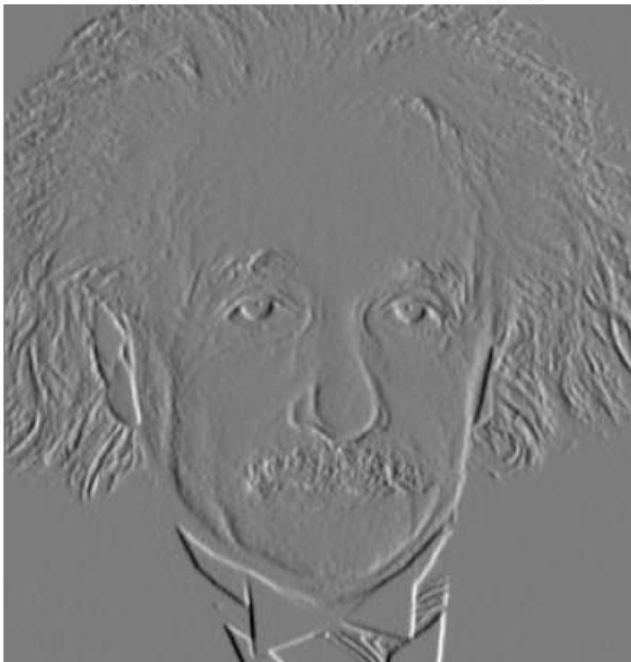
conv1 = cv.filter2D(f, cv.CV_64F, h_kernel)
img_filtered = cv.filter2D(conv1, cv.CV_64F, v_kernel)
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(img_filtered, cmap='gray')
plt.title("Separable 1D Filters (Vertical Edges)")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(img_filtered_horizontal, cmap='gray')
plt.title("Separable 1D Filters (Horizontal Edges)")
plt.axis("off")

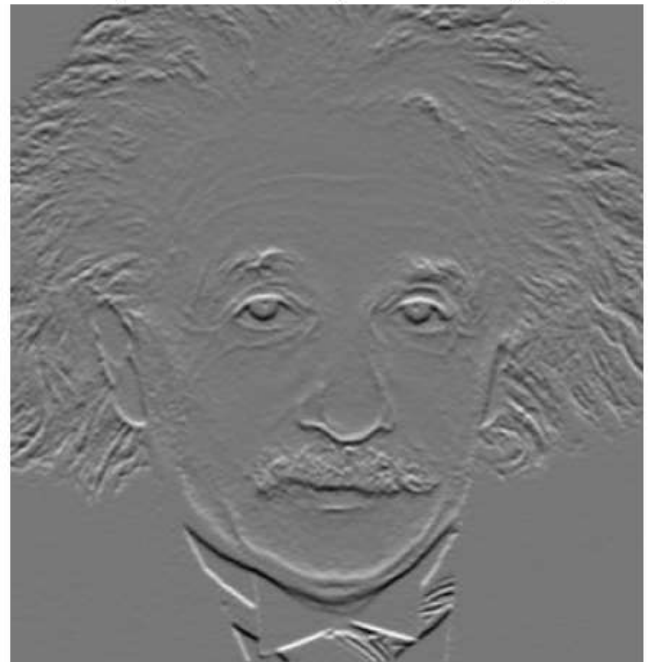
plt.tight_layout()
plt.show()

```

Separable 1D Filters (Vertical Edges)



Separable 1D Filters (Horizontal Edges)



Question 8: Image Scaling and Interpolation Evaluation

```

In [253]: def zoom_image(img, scale_factor, method='nearest'):
            h, w = img.shape[:2]
            new_h = int(h * scale_factor)
            new_w = int(w * scale_factor)
            r_new = np.arange(new_h)
            c_new = np.arange(new_w)
            r_old = r_new / scale_factor
            c_old = c_new / scale_factor

            if method == 'nearest':
                r_idx = np.clip(np.round(r_old).astype(int), 0, h - 1)
                c_idx = np.clip(np.round(c_old).astype(int), 0, w - 1)
                return img[r_idx[:, None], c_idx]

```

```

elif method == 'bilinear':
    img_f = img.astype(np.float64)
    r0 = np.clip(np.floor(r_old).astype(int), 0, h - 1)
    r1 = np.clip(r0 + 1, 0, h - 1)
    c0 = np.clip(np.floor(c_old).astype(int), 0, w - 1)
    c1 = np.clip(c0 + 1, 0, w - 1)
    dr = (r_old - r0).reshape(-1, 1)
    dc = (c_old - c0).reshape(1, -1)
    if len(img.shape) == 3:
        dr = dr.reshape(-1, 1, 1)
        dc = dc.reshape(1, -1, 1)
    Ia = img_f[r0[:, None], c0] # Top-Left
    Ib = img_f[r0[:, None], c1] # Top-Right
    Ic = img_f[r1[:, None], c0] # Bottom-Left
    Id = img_f[r1[:, None], c1] # Bottom-Right
    wa = (1 - dr) * (1 - dc)
    wb = (1 - dr) * dc
    wc = dr * (1 - dc)
    wd = dr * dc
    zoomed_img = Ia * wa + Ib * wb + Ic * wc + Id * wd

    return np.clip(zoomed_img, 0, 255).astype(np.uint8)

else:
    raise ValueError("Method must be 'nearest' or 'bilinear'")

```

```

In [254]: def n_ssd(im1, im2):
    im1 = im1.astype(np.float64)
    im2 = im2.astype(np.float64)
    return np.sum((im1 - im2) ** 2) / np.sum(im2 ** 2)

```

```

In [255]: image_pairs = [
    (r"D:\Semester 5\CV\EN3160_Assignment_01\images\im01small.png",
     r"D:\Semester 5\CV\EN3160_Assignment_01\images\im01.png"),

    (r"D:\Semester 5\CV\EN3160_Assignment_01\images\im02small.png",
     r"D:\Semester 5\CV\EN3160_Assignment_01\images\im02.png")
]

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

scale_factor = 4

for i, (path_small, path_large) in enumerate(image_pairs):
    img_small = cv.cvtColor(cv.imread(path_small, cv.IMREAD_COLOR), cv.COLOR_BGR2RGB)
    img_large = cv.cvtColor(cv.imread(path_large, cv.IMREAD_COLOR), cv.COLOR_BGR2RGB)

    img_nn = zoom_image(img_small, scale_factor, method='nearest')
    img_bl = zoom_image(img_small, scale_factor, method='bilinear')
    ssd_nn = n_ssd(img_nn, img_large)
    ssd_bl = n_ssd(img_bl, img_large)

    axes[i, 0].imshow(img_large)
    axes[i, 0].set_title(f"Pair {i+1}: Original High-Res")
    axes[i, 0].axis("off")

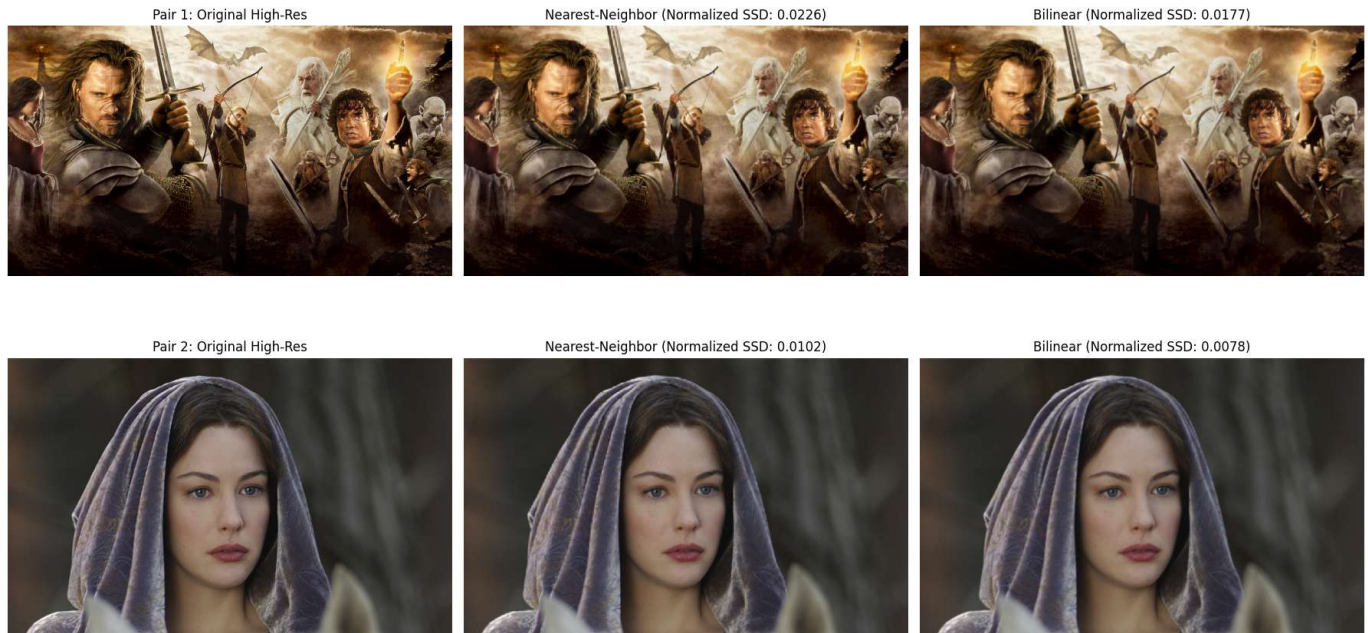
    axes[i, 1].imshow(img_nn)
    axes[i, 1].set_title(f"Nearest-Neighbor (Normalized SSD: {ssd_nn:.4f})")
    axes[i, 1].axis("off")

    axes[i, 2].imshow(img_bl)
    axes[i, 2].set_title(f"Bilinear (Normalized SSD: {ssd_bl:.4f})")

```

```
axes[i, 2].axis("off")
```

```
plt.tight_layout()
plt.show()
```



8. Discussion: Interpolation Methods Evaluation

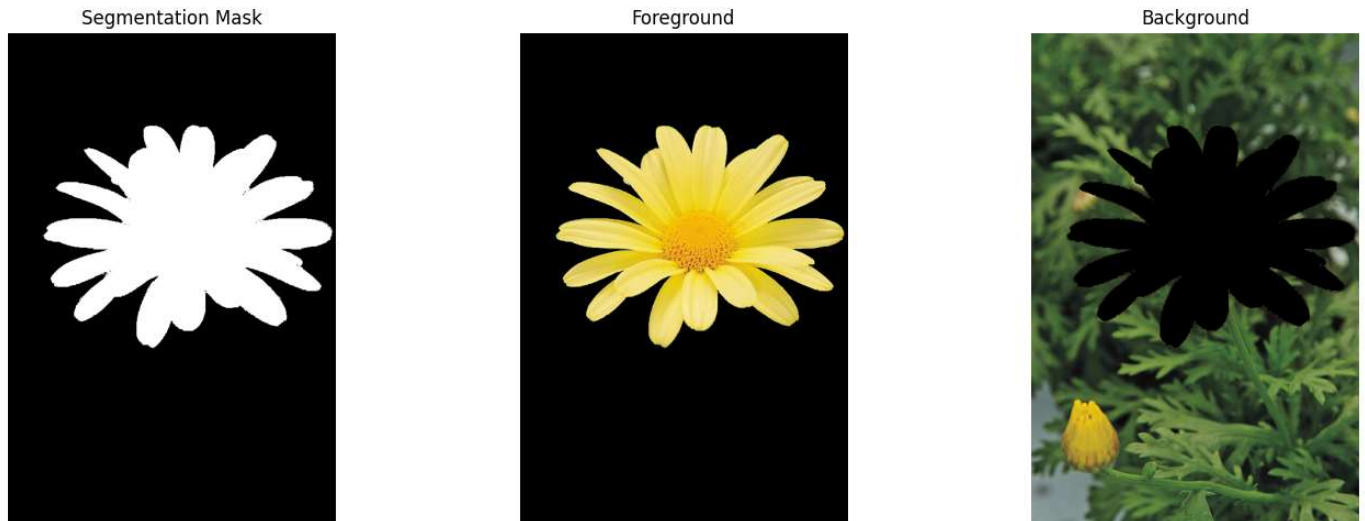
Based on the visual outputs and the computed Normalized Sum of Squared Differences (SSD) for both image pairs, we can observe the following:

- For both, bilinear interpolation have yielded a lower Normalized SSD compared to Nearest-Neighbor (0.0177 vs 0.0226 for Pair 1, and 0.0078 vs 0.0102 for Pair 2). A lower SSD proves that the Bilinear upscaling method reconstructs an image that is significantly closer to the original high resolution image.
- Nearest Neighbor interpolation works from duplicating the closest original pixel and computationally fast. This create aliasing, pixelation when scaling up by a large factor.
- Bilinear interpolation computes a distance weighted average of four closest surrounding pixels. This creates smoother gradients, more natural transitions. So it mathematically and visually outperforms Nearest Neighbor when enlarging.

Question 9: GrabCut Segmentation and Depth-of-Field Enhancement

```
In [256]: img_flower = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\daisy.jpg")
img_rgb = cv.cvtColor(img_flower, cv.COLOR_BGR2RGB)
mask = np.zeros(img_flower.shape[:2], np.uint8)
bgdModel = np.zeros((1, 65), np.float64)
fgdModel = np.zeros((1, 65), np.float64)
rect = (20, 130, 535, 470)
cv.grabCut(img_flower, mask, rect, bgdModel, fgdModel, 5, cv.GC_INIT_WITH_RECT)
mask_binary = np.where((mask == 2) | (mask == 0), 0, 1).astype('uint8')
foreground = cv.bitwise_and(img_rgb, img_rgb, mask=mask_binary)
mask_inv = cv.bitwise_not(mask_binary * 255) # Invert mask for background
background = cv.bitwise_and(img_rgb, img_rgb, mask=mask_inv)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(mask_binary, cmap='gray'); axes[0].set_title("Segmentation Mask"); axes[0].axis(
```

```
axes[1].imshow(foreground); axes[1].set_title("Foreground"); axes[1].axis('off')
axes[2].imshow(background); axes[2].set_title("Background"); axes[2].axis('off')
plt.tight_layout()
plt.show()
```



```
In [257]: blurred_img = cv.GaussianBlur(img_rgb, (51, 51), 0)
mask_3d = mask_binary[:, :, np.newaxis]
enhanced_img = np.where(mask_3d == 1, img_rgb, blurred_img)
fig, axes = plt.subplots(1, 2, figsize=(12, 6))
axes[0].imshow(img_rgb); axes[0].set_title("Original Image"); axes[0].axis('off')
axes[1].imshow(enhanced_img); axes[1].set_title("Enhanced (Portrait Mode)"); axes[1].axis('off')
plt.tight_layout()
plt.show()
```



9(c) Discussion

The dark region beyond the edge of the flower in the enhanced image is an issue caused by the spatial filtering process. It's called the "color bleeding" or "halo effect".

When the heavy Gaussian blur is applied to the original image to create the artificial depth of field and the filter calculates weighted average of neighboring pixels. Along the boundary of the flower, the dark pixels from the flower's edges and internal shadows are averaged together with the adjacent lighter background pixels. This spatial mixing artificially darkens the background pixels immediately surrounding the flower.

So when the sharp, segmented foreground mask is superimposed back onto this blurred image, the darkened background pixels outside the segmentation boundary remain visible leading to dark halo effect.

Question 10: Edge-Preserving Smoothing via Bilateral Filtering

```
In [258]: img_lake = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\minaret.jpg", cv.IMREAD_COLOR)
img_rgb = cv.cvtColor(img_lake, cv.COLOR_BGR2RGB)
img_gaussian = cv.GaussianBlur(img_rgb, (9, 9), sigmaX=1)
img_bilateral_cv = cv.bilateralFilter(img_rgb, d=9, sigmaColor=35, sigmaSpace=70)
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
axes[0].imshow(img_rgb); axes[0].set_title("Original Image"); axes[0].axis('off')
axes[1].imshow(img_gaussian); axes[1].set_title("Gaussian Blur"); axes[1].axis('off')
axes[2].imshow(img_bilateral_cv); axes[2].set_title("OpenCV Bilateral"); axes[2].axis('off')
plt.tight_layout()
plt.show()
```



```
In [259]: def custom_bilateral_filter_channel(img, d, sigma_color, sigma_space):
    """Applies bilateral filter to a single 2D channel."""
    img = img.astype(np.float64)
    rows, cols = img.shape
    output = np.zeros((rows, cols), dtype=np.float64)
    r = d // 2
    x, y = np.meshgrid(np.arange(-r, r+1), np.arange(-r, r+1))
    spatial_weights = np.exp(-(x**2 + y**2) / (2 * sigma_space**2))
    padded_img = np.pad(img, pad_width=r, mode='reflect')
    for i in range(rows):
        for j in range(cols):
            patch = padded_img[i:i+d, j:j+d]
            center_val = padded_img[i+r, j+r]
            range_weights = np.exp(-((patch - center_val)**2) / (2 * sigma_color**2))
            total_weights = spatial_weights * range_weights
            output[i, j] = (np.sum(total_weights * patch) / np.sum(total_weights)).astype(np.float64)

    return output
```

```
def custom_bilateral_color(img_rgb, d, sigma_color, sigma_space):
    """Splits color image, filters channels, and merges."""
    r, g, b = cv.split(img_rgb)
    r_filtered = custom_bilateral_filter_channel(r, d, sigma_color, sigma_space)
    g_filtered = custom_bilateral_filter_channel(g, d, sigma_color, sigma_space)
    b_filtered = custom_bilateral_filter_channel(b, d, sigma_color, sigma_space)

    merged = cv.merge([r_filtered, g_filtered, b_filtered])
    return np.clip(merged, 0, 255).astype(np.uint8)
```

```
In [260]: img_bilateral_custom = custom_bilateral_color(img_rgb, d=9, sigma_color=35, sigma_space=70)
normalized_ssd = n_ssd(img_bilateral_custom, img_bilateral_cv)
print(f"Normalized SSD (OpenCV vs Custom): {normalized_ssd:.4f}")

fig, axes = plt.subplots(1, 4, figsize=(24, 6))
axes[0].imshow(img_rgb)
axes[0].set_title("Original Image", fontsize=14)
axes[0].axis('off')
axes[1].imshow(img_gaussian)
axes[1].set_title("Gaussian Blur", fontsize=14)
axes[1].axis('off')
axes[2].imshow(img_bilateral_custom)
axes[2].set_title("Custom Bilateral Filter", fontsize=14)
axes[2].axis('off')
axes[3].imshow(img_bilateral_cv)
axes[3].set_title(f"OpenCV Bilateral (Normalized SSD vs Custom: {normalized_ssd:.4f})", fontsize=14)
axes[3].axis('off')

plt.tight_layout()
plt.show()
```

Normalized SSD (OpenCV vs Custom): 0.0053



10(b) Discussion: Custom vs. OpenCV Bilateral Filter

Both custom and OpenCV bilateral filters successfully eliminate visible noise in the water while preserving sharp structural edges and shows a clear advantage compared to the uniform blurring of the standard Gaussian filter. The custom implementation give a highly accurate Normalized SSD of 0.0053 against OpenCV's algorithm. This minor errors arise because the function processes RGB channels independently such that OpenCV utilizes a unified color vector to calculate range weights.